

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гродненский государственный университет имени Янки Купалы»

Факультет математики и информатики
Кафедра системного программирования и компьютерной безопасности

ЕВЧАР ЕГОР АНАТОЛЬЕВИЧ

Разработка программных средств для анализа движения автомобильного транспорта на основе данных потокового видео

Дипломная работа

студента 4 курса дневной формы получения образования

Специальность 1-40 03 01 «Искусственный интеллект»

Руководитель:

Вувуникян Юрий
Микиртычевич
профессор кафедры
СПиКБ, доктор
физико-
математических наук,
профессор

Студент:

Допущен к защите _____ 2024 г.

Заведующий кафедрой,
кандидат технических наук, доцент:

_____ / Кадан А.М

Гродно, 2024

РЕЗЮМЕ

Дипломная работа: 53 с., 7 рис., 8 источника.

НЕЙРОННЫЕ СЕТИ, ГЛУБОКОЕ ОБУЧЕНИЕ, СВЕРТОЧНЫЕ НЕЙРОННЫЕ СЕТИ, ТЕХНОЛОГИИ КОМПЬЮТЕРНОГО ЗРЕНИЯ, КЛАССИФИКАЦИЯ ИЗОБРАЖЕНИЙ, СЕГМЕНТАЦИЯ ИЗОБРАЖЕНИЙ, ДЕТЕКЦИЯ ОБЪЕКТОВ

Объект исследования: нейронная сеть.

Предмет исследования: нейронная сеть для анализа движения автомобильного транспорта по видео потоку.

Цель работы: реализовать рабочую нейронную сеть для анализа движения автомобильного транспорта по видео потоку.

Методы исследования: метод обучения с учителем.

Исследования и разработки: изучены методы обучения нейронной сети.

Элементы научной новизны: предложены подходы к обучению нейронной сети для анализа движения автомобильного транспорта по видео потоку.

Область возможного практического применения: область возможного практического применения включает в себя использование разработанных программных средств для улучшения транспортной инфраструктуры городов, оптимизации управления дорожным движением, а также для повышения безопасности на дорогах. Эти программные средства могут быть интегрированы в интеллектуальные транспортные системы, что позволит автоматизировать мониторинг трафика, а также оптимизировать управление светофорами и другими дорожными сооружениями. Кроме того, они могут быть использованы для предоставления водителям и дорожным службам актуальной информации о состоянии дороги в реальном времени, способствуя более эффективному планированию маршрутов и реагированию на изменения в транспортной ситуации.

Автор работы подтверждает, что приведённый в дипломной работе расчётно-аналитический материал правильно и объективно отражает состояние исследуемого процесса, а все заимствованные из литературных и других источников теоретические, методологические и методические положения и концепции сопровождаются ссылками на их авторов.

SUMMARY

Graduate work: 53 pp, 7 fig., 8 ref.

NEURAL NETWORKS, DEEP LEARNING, CONVOLUTIONAL NETWORKS, NEURAL NETWORKS, COMPUTER VISION TECHNOLOGIES, IMAGE CLASSIFICATION, IMAGE SEGMENTATION, IMAGE DETECTION, DETECTION OF CARS IN IMAGES. COMPUTER VISION, IMAGE CLASSIFICATION, IMAGE SEGMENTATION, OBJECTS DETECTION.

The object of the research: neural network.

The subject of the research: neural network for analyzing the movement of road transport using a video stream.

The purpose of the research: to implement a working neural network for analyzing the movement of road transport using a video stream.

neural network for recognizing car brands and models from a photo.

The research methods: the method of learning with the teacher.

Research and development: neural network training methods are studied.

The elements of scientific novelty: approaches to training a neural network for analyzing the movement of road transport using a video stream are proposed.

The area of possible practical application: The area of possible practical application includes the use of developed software to improve the transport infrastructure of cities, optimize traffic management, and also to improve road safety. These software tools can be integrated into intelligent transport systems, which will automate traffic monitoring, as well as optimize the management of traffic lights and other road structures. In addition, they can be used to provide drivers and traffic authorities with up-to-date information about road conditions in real time, facilitating more efficient route planning and response to changes in traffic conditions.

The author of work confirms that the design and analytical materials of graduate work reflect the state of the process under research correctly and impartially, and all theoretical, methodological and methodical statements and conceptions borrowed from the literature and other sources are accompanied by references to their authors.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. ВВЕДЕНИЕ В КОМПЬЮТЕРНОЕ ЗРЕНИЕ И РАСПОЗНАВАНИЕ ИЗОБРАЖЕНИЙ.....	7
1. История и развитие компьютерного зрения.....	7
2. Основные концепции и методы компьютерного зрения ...	12
3. Основные техники обработки изображений	14
4. Методы распознавания изображений	16
1.5. Машинное обучение и глубокое обучение в компьютерном зрении.....	18
1.6. Выводы по главе 1	20
ГЛАВА 2. ПОСТАНОВКА ЗАДАЧИ.....	21
2.1. Актуальность задачи.....	21
2.2. Цель и задачи исследования	22
2.3 Ожидаемые результаты	23
2.4. Выводы по главе 2.....	24
ГЛАВА 3. РЕШЕНИЕ ЗАДАЧИ ПОСТРОЕНИЯ НЕЙРОННОЙ СЕТИ ДЛЯ АНАЛИЗА ДВИЖЕНИЯ АВТОМОБИЛЬНОГО ТРАНСПОРТА.....	24
3.1. Реализация системы для анализа движения автомобильного транспорта на основе данных потокового видео	24

3.2. Выводы по главе 3.....	41
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42

ВВЕДЕНИЕ

Для управления городской инфраструктурой и повышения безопасности на дорогах требуется анализировать городской трафик. Для решения этой задачи можно использовать различные датчики, информацию с облачных сервисов, таких как Google Maps, а также технологии компьютерного зрения. Они помогут предоставлять данные о загруженности дорог, о скорости движения транспорта и аварийных ситуациях.

За последнее время технологии компьютерного зрения и машинного обучения получили значительное улучшение качества детекции, классификации и сегментации данных. Применение технологий компьютерного зрения позволяет автоматически обнаруживать, классифицировать и отслеживать трафик без помощи каких-либо датчиков и облачных сервисов. Применение алгоритмов глубокого обучения позволяет получить высокую точность при распознавании пешеходов, различных типов транспорта и анализе их движения, а с развитием мощных видеокарт и программных платформ эти технологии стали доступнее.

Анализ трафика по видеопотоку встраивается в интеллектуальные транспортные системы, тем самым позволяя автоматизировать мониторинг трафика, оптимизировать управление светофорами и предоставлять информацию о состоянии дороги в реальном времени водителям и дорожным организациям. Также такие системы могут объединяться с другими системами умного города.

Преимуществом анализа городского трафика с использованием технологий компьютерного зрения является независимость от различных

датчиков и прочих систем – для этого достаточно будет одной камеры видеонаблюдения. Они могут устанавливаться на перекрестках, вдоль дорог и на других значимых местах. Из недостатков можно выделить необходимость обработки больших объемов данных и требования к вычислительным ресурсам, но с развитием облачных вычислений эти проблемы становятся не такими серьезными, потому как данные могут обрабатываться в облаке, а это в свою очередь снижает нагрузку на локальные системы.

Анализ городского трафика по видеопотоку с использованием технологий компьютерного зрения позволяет улучшить управление городской инфраструктурой и повысить эффективность транспортной системы. Дальнейшее развитие этого решения улучшит интеллектуальные транспортные системы и адаптивные системы управления транспортом. Данная работа направлена на создание решения, способного решить проблемы городского трафика, такие как пробки и аварийные ситуации, а это в свою очередь повысит уровень безопасности и улучшит экологическую ситуацию.

В теории, встраивание систем анализа движения транспорта на основе потокового видео должно сократить время реагирования на дорожные происшествия, тем самым снизит негативное влияние на трафик и обеспечит плавное и безопасное движение на дорогах.

ГЛАВА 1. ВВЕДЕНИЕ В КОМПЬЮТЕРНОЕ ЗРЕНИЕ И РАСПОЗНАВАНИЕ ИЗОБРАЖЕНИЙ

1. История и развитие компьютерного зрения

Компьютерное зрение – одно из основных направлений в машинном обучении. Его цель – это научить компьютеры понимать визуальную информацию из окружающего мира так, как это делает мозг человека.

Появление: 1950–1970

История компьютерного зрения начинается в 1950-1970 годах. В 1957 году был изобретен цифровой сканер, который позволял преобразовывать печатные изображения в цифровой формат. В 1959 году было изучение зрительной коры головного мозга, которое дало ценную информацию о том, как живые существа воспринимают визуальную информацию. В 1963 году началась работа над реконструкцией формы 3D моделей. В 1966 году был запущен проект "The Summer Vision Project", цель которого была заставить машины понимать окружающий мир.

Классические алгоритмы CV: 1970–1990

С 1970 по 1990 годы в области компьютерного зрения было разработано множество классических алгоритмов, используемых по сей день. В 1981 году была предложена методика оценки оптического потока, позволяющая отслеживать движение объектов на изображении. В 1982 году была сформулирована теория анализа визуальной информации. В 1986 году появился детектор границ Кэнни, который до сих пор используется для выделения контуров объектов. В 1991 году было представлено распознавание

лиц с использованием собственных векторов. В 1998 году были применены свёрточные нейронные сети CNN для распознавания рукописных цифр. Методы Виолы Джонса для поиска лиц созданные в 2001 году и алгоритм SURF для быстрого вычисления локальных признаков созданные в 2006 году стали прорывом в этой области.

Области применения

Поиск объектов встречается сейчас повсюду. Начиная от использования масок в социальных сетях, заканчивая поиском лиц через камеры видеонаблюдения. Видеонаблюдение с использованием компьютерного зрения автоматически обнаруживает и отслеживает объекты или действия в реальном времени. Технологии поиска объектов используются в беспилотных автомобилях для распознавания дорожных знаков, пешеходов и других транспортных средств, обеспечивая безопасность и автономность передвижения.

Распознавание лиц стало самой обсуждаемой темой в технологическом сообществе. Оно широко используется в системах безопасности для идентификации и аутентификации личности. Эту технологию можно встретить в устройствах при его разблокировке, таких как телефон и ноутбук. В социальных сетях эта технология используется для наложения различных масок на лицо, тегирования фотографий и прочего.

Глубокое обучение: с 2010

Революция в компьютерном зрении произошла в 2010-х годах с появлением глубокого обучения. В 2012 году модель AlexNet совершила прорыв в распознавании изображений на конкурсе ImageNet, что стало началом новой эры. В 2014 году была представлена модель VGG, показавшая, что увеличение числа слоев нейронной сети может сильно повысить качество распознавания. В 2015 году архитектура ResNet с использованием остаточных блоков создала сети, которые превосходят человеческие возможности в некоторых задачах распознавания.

Основные причины начала новой эры

Несмотря на то, что все необходимые математические инструменты были разработаны еще к середине 1980-х годов, революция в компьютерном

зрении произошла только в 2012 году. Это стало возможным благодаря нескольким причинам: появлению больших наборов данных для обучения, развитию мощных видеокарт, которые значительно ускорили процесс обучения нейронных сетей, и открытому доступу к исследованиям через платформы, такие как arXiv и GitHub.

Основные направления компьютерного зрения

Компьютерное зрение содержит в себе множество направлений, каждое из которых заточено под решение определенной задачи. Рассмотрим поближе каждое из направлений:

Классификация

Одной из наиболее изученных задач в компьютерном зрении является классификация. Классификация изображений — это направление, решающее к какому классу принадлежит объект на изображении. Например, модель классификации может определить кто изображен на фотографии: кошка или собака. Классификация изображений лежит в основе многих приложений. Также классификация используется в системах безопасности, например в смартфонах она помогает в распознавании лиц.

Примеры моделей для классификации и их год появления:

- 2012 — AlexNet
- 2014 — VGGNet
- 2014 — Inception
- 2015 — ResNet
- 2016 — ResNeXt
- 2017 — MobileNet
- 2017 — (P)NASNet
- 2019 — EfficientNet
- 2020 — ViT
- 2021 — Swin Transformer
- 2022 — ConvNeXts

Сегментация

Сегментация изображений решает проблему разделения изображения на значимые части или объекты. Например, у нас есть фотография, где изображены кот и собака, сегментация в данном случае покажет область пикселей, где находится кот и отдельно покажет, где находится собака. Существуют два основных типа сегментации: семантическая и инстанс-сегментация. Семантическая сегментация присваивает каждому пикселю на изображении метку, которая определяет к какому классу он принадлежит (из нашего примера, кот, собака или лошадь). Инстанс-сегментация, помимо определения класса, различает отдельные объекты одного и того же класса, например сиамский кот и бенгальский кот принадлежат одному классу кот.

Примеры моделей для сегментации и их год появления:

2014 — FCN
2015 — UNet
2015 — SegNet
2016 — RefineNet
2016 — PSPNet
2017 — Mask RCNN
2018 — DeepLabv3+
2020 — ResNeSt
2020 — DetectoRS
2021 — SETR

Детекция

Следующее важное направление — это детекция. Детекция объектов решает задачу определения и нахождения положения объектов внутри изображения. В отличие от классификации, детекция помимо определения наличия объекта указывает его местоположение с помощью ограничивающего прямоугольника, называемого bounding box. В пример детекции можно привести обнаружение пешеходов на дорогах и распознавание автомобилей для анализа трафика. Для систем, требующих понимания окружающей среды в реальном времени детекция самый важный компонент.

Примеры моделей для детекции и их год появления:

2013 — Overfeat

2014 — RCNN
2015 — Faster RCNN
2015 — YOLO
2015 — SSD
2016 — RFCN
2017 — FPN
2017 — RetinaNet
2019 — CenterNet
2019 — EfficientDet
2020 — DETR

Multi-view Geometry (Многокамерная геометрия)

Multi-view geometry еще одно направление в компьютерном зрении, созданное для восстановления трехмерной структуры объекта путем анализа изображений, полученных с нескольких камер. Многокамерная геометрия может создавать точные 3D модели объектов, 3D карты и реконструкции.

Трекинг и видеоаналитика

Трекинг объектов занимается отслеживанием положения и передвижения объектов на кадрах видео. Это направление будет использоваться в данной работе для отслеживания передвижения каждого транспортного средства в потоковом видео.

Биометрия

Биометрия это направление, созданное для идентификации людей по их физиологическим свойствам, таких как лицо. Помимо самой популярной технологии распознавания лиц, существуют такие способы идентификации как отпечатки пальцев, радужная оболочка глаза и голос. Эти способы часто применяются в системах безопасности, банковских услугах и мобильных устройствах для идентификации пользователей. У биометрии высокий уровень безопасности, поскольку у каждого человека уникальные физиологические характеристики.

Оценка качества изображений

Следующее направление, это оценка качества изображений. Оно анализирует изображения и определяет их качество путем измерения различных параметров, таких как четкость, яркость, цветовая точность и уровень шума. Это направление полезно в приложениях, связанных с фотографией. Оно облегчает выбор снимков, помогает в процессе съемки и помогает обрабатывать изображения.

Улучшение и восстановление изображений

Данное направление служит для улучшения визуальных характеристик изображения, таких как повышение качества изображения, повышение четкости и удаления цветковых шумов. Также это направление способно восстанавливать фотографии, будь то поврежденные или утраченные части фотографии.

Построение карты глубины

Это направление нацелено на получение изображения, в котором вместо цвета пикселя хранится расстояние до камеры. Оно часто применяется в умных устройствах, таких как робот-пылесос, транспортных средствах с автопилотом и в устройствах дополненной реальности для ориентирования в пространстве. Например, эти карты дают роботу-пылесосу точное расстояние до объектов и помогают избегать столкновений с ними.

2. Основные концепции и методы компьютерного зрения

Компьютерное зрение построено на разных методах, которые дают компьютерам возможность анализировать и интерпретировать визуальную информацию. Рассмотрим подробнее каждые из них, которые лежат в основе многих приложений и систем.

Свёрточные нейронные сети (CNN)

Свёрточные нейронные сети (Convolutional Neural Networks, CNN) – это самый известный и популярный метод анализа изображений в

компьютерном зрении. Он состоит из слоев сверток, объединений называемых pooling и полностью связанных слоев. Они сами учатся выделять признаки из изображений, по типу краев, текстур и более сложных структур. Техническим прорывом стала сеть AlexNet выпущенная в 2012 году, которая показала свои способности на конкурсе ImageNet.

Оптический поток

Оптический поток — это способ определить движение объектов в видео, глядя на изменения в яркости пикселей между кадрами. Этот метод помогает следить за движением, анализировать видео и стабилизировать изображение. Он может отслеживать как машины едут по дороге или как люди идут по тротуару.

Метод главных компонент (PCA)

Этот метод используется для упрощения данных и выделения самых важных признаков. В компьютерном зрении PCA помогает распознавать лица и классифицировать объекты, уменьшая количество данных и ускоряя их обработку. Его часто применяют для предварительной обработки данных перед использованием более сложных моделей.

Машинное обучение и классификация

Методы машинного обучения, такие как Support Vector Machines, Random Forests и K-Nearest Neighbors, используются в компьютерном зрении для классификации. Они помогают моделям учиться на размеченных данных и предсказывать что изображено на новых снимках.

Детерминированные и стохастические методы оптимизации

Оптимизация, такая как градиентный спуск, помогает уменьшить ошибки в нейронных сетях. Стохастический градиентный спуск полезен для работы с большими объемами данных. Эти два метода помогают моделям находить подходящие настройки и улучшают их работу.

Глубокие рекуррентные сети (RNN)

Глубокие рекуррентные сети и их улучшенные версии, такие как LSTM и GRU, используются для работы с последовательными данными такими как

видео. Они учитывают контекст и связи во времени, запоминают информацию о предыдущих кадрах и помогают отслеживать объекты, предсказывать последовательности и анализировать временные ряды.

Аугментация данных

Аугментация данных — это способ увеличения тренировочных данных путем их изменения. Это может быть вращение, изменение размера, обрезка и изменение яркости изображений. Аугментация помогает моделям лучше справляться с различными изменениями в данных.

Обратное распространение ошибки (Backpropagation)

Обратное распространение ошибки — это способ обучения нейронных сетей. Он помогает вычислять насколько каждый вес в сети должен быть изменен чтобы уменьшить ошибки. Этот метод позволяет улучшать точность моделей. С его помощью современные модели стали гораздо точнее.

Методы регуляризации

Регуляризация помогает моделям не запоминать слишком сильно тренировочные данные, чтобы они могли хорошо работать с новыми данными. Методы, такие как L1 и L2 регуляризация, Dropout и Batch Normalization, делают модели устойчивыми и надежными, что важно для обучения глубоких нейронных сетей чтобы они лучше справлялись с новыми данными.

Эти идеи и методы лежат в основе компьютерного зрения.

3. Основные техники обработки изображений

Обработка изображений помогает изменять и изучать изображения для использования в разных программах. Основные техники включают изменение изображений, извлечение особенностей и особые операции.

Преобразование изображений

Преобразование изображений включает в себя много способов изменения или улучшения картинок перед тем, как их анализировать.

Фильтрация, одна из главных техник, которая помогает убрать шум, повысить контраст и выделить важные части изображения. Фильтрация бывает простая и более сложная. Каждый вид фильтрации имеет свои нюансы и используется в зависимости от поставленных задач.

Линейная фильтрация это когда применяются особые фильтры к изображению чтобы его изменить. Эти фильтры работают на основе окружающих пикселей. Гауссово размытие делает изображение менее резким, а фильтр повышения резкости делает его более детальным. Нелинейные фильтры, такие как медианный фильтр, используются для удаления шумов, но сохраняют важные детали.

Свертка это процесс прохождения специального окна по каждому пикселю на картинке, после чего значения пикселей перемножаются с соответствующими значениями в этом окне и складываются результаты. Этот процесс находит разные особенности на картинке, такие как края или текстуры.

Преобразование Фурье еще один способ анализа изображения, который помогает понять какие частоты присутствуют на картинке. Он используется чтобы убирать шум и сжимать изображения.

Извлечение признаков

Процесс выявления важных частей и информации на картинке, таких как края и углы называется извлечением признаков. Эти части используются для определения объектов и анализа того, что изображено на изображении. Этот процесс происходит с помощью специальных операций, которые ищут изменения в яркости между соседними пикселями и показывают, где находятся границы объектов.

Детекторы контуров, как метод Кэнни, делают несколько шагов для поиска контуров на изображении. Для начала они убирают шум, затем находят изменения яркости на картинке. После этого они применяют фильтры для выделения самых ярких линий и отслеживают их.

Метод Харриса помогает найти углы, а также места, где линии касаются друг друга. Он использует математические вычисления, чтобы определить, где на изображении есть углы. Этот метод помогает при сравнении изображений и создании трехмерных моделей сцен.

Дескрипторы признаков, такие как SIFT и SURF, помогают находить и описывать особенности на изображениях. Они это делают специальным образом, чтобы признаки оставались узнаваемыми, даже если изображение видоизменяется. Эти методы изучают изображения при разных масштабах и находят ключевые точки, описывая их особенности с помощью векторных дескрипторов. Дескрипторы используются для сравнения и нахождения совпадений между разными изображениями.

4. Методы распознавания изображений

Как было сказано ранее, методы распознавания изображений включают в себя такие направления как классификация и детекция объектов, сегментация изображений и методики отслеживания объектов. Эти направления предназначены для определенных задач и построены на новейших алгоритмах и моделях машинного и глубокого обучения.

Классификация и детекция объектов

Классификация объектов это процесс присвоения изображениям ярлыков из списка категорий, другими словами, компьютер учится различать объекты по их внешнему виду. Компьютер можно научить отличать кошек от собак на фотографиях. За моделью AlexNet последовали более сложные модели, такие как VGGNet, Inception, ResNet и EfficientNet. Они стали еще лучше в распознавании объектов, потому что стали глубже и использовали разные типы слоев.

Детекция объектов же более сложный процесс, чем просто определение, что изображено на картинке. Она включает нахождение местоположения объектов на изображении. Среди первых выделяется R-CNN, которая работала неплохо, но была медленной, потому что нужно было анализировать каждую рамку отдельно. Затем появились улучшенные версии, такие как Fast R-CNN и Faster R-CNN, они стали быстрее за счёт использования особых сетей и методов, которые помогают быстрее определить, где находятся объекты на картинке.

YOLO и SSD две популярные модели для распознавания объектов на изображениях, одна из которых будет использоваться в данном проекте. Они позволяют моделям быстро находить объекты и определять, к какому классу

они относятся. YOLO делает это за один проход, а значит является идеальным решением для использования в системах реального времени. SSD также эффективен, используя несколько размеров областей для поиска объектов. Обе модели сделали прорыв в детекции объектов и стали лидирующими моделями, которые используются во многих системах.

Именно YOLO будет использоваться в создании программного средства для анализа трафика, поскольку она имеет высокую точность и скорость детекции.

Сегментация изображений

Сегментация изображений решает задачу деления картинки на части, каждая из которых соответствует отдельному объекту или области. В отличие от детекции, где достаточно рамкой показать, где находится объект, надо очень точно определить границы каждого объекта. Сегментация бывает двух видов: семантическая и инстансная. Первая классифицирует каждый пиксель изображения как какой-то класс, не учитывая отдельные экземпляры объектов, а вторая кроме классификации каждого пикселя учитывает разные экземпляры объектов того же класса.

Первым успешным методом семантической сегментации была Fully Convolutional Network, которая заменила полностью связанные слои свёрточной сети на сверточные, что позволило сети принимать изображения любого размера и генерировать карты классов пикселей. Он стал основой для таких моделей, как U-Net.

Примерами более сложных моделей, которые улучшили сегментацию за счет использования пирамидальных пуллинговых модулей и методов пространственного пирамидального пула являются PSPNet и DeepLab.

Для задачи инстанс сегментации была создана Mask R-CNN, которая расширила Faster R-CNN, добавив дополнительную ветку для предсказания масок объектов на пиксельном уровне, показав высокую точность и став стандартом для многих приложений, требующих точного определения и различения объектов.

Методики отслеживания объектов

Optical Flow один из методов, который позволяет увидеть, как пиксели двигаются между кадрами в видео. Он помогает стабилизировать видео,

понять, как двигается что-то на экране и отслеживать объекты в видео. Есть два основных способа как это сделать: метод Лукаса-Канаде и метод Хорн-Шунка. Они используют разные подходы к определению движения, полагаясь на то, как меняется яркость пикселей.

Корреляционные методы используют шаблоны объектов и ищут их в последующих кадрах. Корреляционные фильтры, такие как Kernelized Correlation Filters, помогают отслеживать объекты даже если они меняют масштаб, поворачиваются или деформируются.

Есть метод GOTURN, который использует сети, основанные на сверточных нейронных сетях, чтобы предсказывать, где будет находиться объект на следующем кадре основываясь на текущем кадре и его предыдущем положении.

Классификация объектов, определение их на изображениях, разделение изображений на отдельные части и отслеживание объектов постоянно развиваются. Все эти методы решают многие задачи, начиная от автопилотов и заканчивая видеонаблюдением и безопасностью.

1.5. Машинное обучение и глубокое обучение в компьютерном зрении

Компьютерное зрение очень связано с машинным и глубоким обучением. Машинное обучение это процесс обучения компьютеров, когда они учатся на данных и становятся лучше в выполнении некоторых задач. Изначально для этого использовались классические алгоритмы, типа линейной регрессии, метода опорных векторов и дерева решений. Они помогли достичь первых успехов в компьютерном зрении, но потом главную роль заняли нейронные сети.

Основы машинного обучения включают в себя методы, такие как обучение с учителем, обучение без учителя и обучение с подкреплением.

Процесс обучение с учителем – это способ обучения, при котором компьютер учится на размеченных данных. Метод k-ближайших соседей и наивный байесовский классификатор использовались для распознавания образов, но имели ограничения в точности и обработке сложных данных.

С появлением нейронных сетей произошел большой скачок в машинном обучении. Их работа похожа на работу мозга: у них есть слои с искусственными нейронами, которые обрабатывают информацию. Первоначально были однослойные нейронные сети, которые не могли обучаться сложным вещам из-за их малой размерности. Теперь появились многослойные сети, которые способны учиться, в прошлом их использование было ограничено из-за ограничений вычислительных ресурсов и слабыми алгоритмами обучения.

Применение нейронных сетей в компьютерном зрении сильно увеличилось с появлением глубоких свёрточных нейронных сетей. Их основная задача заключается в автоматическом поиске особенностей на изображениях. CNN состоят из свёрточных слоёв, слоёв объединения и полносвязных слоёв. Свёрточные слои применяют фильтры для выделения разных признаков. Слои объединения уменьшают размер данных оставляя важные особенности, позволяя уменьшить сложность вычислений и предотвратить избыточное обучение.

В 1998 году Ян Лекун создал модель под названием LeNet. Она была хороша для распознавания написанных от руки цифр. Далее вышла в свет модель AlexNet, которая в отличие от LeNet была глубже и имела больше свёрточных слоев и слоев объединения. К тому же в ней использовали технику под названием dropout чтобы избежать переобучения.

Модели, такие как VGGNet и Inception, продолжили этот путь, углубляя архитектуру и внедряя новые методы свёртки и объединения. VGGNet использовала простые и последовательные свёрточные слои, тем самым улучшив процесс обучения и результаты. Inception ввела идею использования свёрточных фильтров разного размера в одном слое, что позволило модели улавливать признаки на различных масштабах и улучшило точность классификации.

Архитектура ResNet, которая была представлена в 2015 году, стала следующим этапом развития глубокого обучения. Основная идея ResNet заключалась в использовании остаточных блоков, которые позволяли создавать очень глубокие нейронные сети с большим количеством слоев, не сталкиваясь с проблемами исчезающего градиента, что позволяло добиться высокой точности и эффективности в задачах распознавания и классификации изображений.

Актуальные модели, такие как YOLO и Faster R-CNN, используются во многих проектах и стали стандартами в области детекции объектов. YOLO преобразовала задачу детекции объектов в задачу регрессии, предсказывая координаты ограничивающих рамок и классы объектов напрямую из изображения, что позволило значительно увеличить скорость детекции, сделав YOLO одной из самых быстрых и точных моделей для реального времени. Faster R-CNN улучшила предыдущие версии R-CNN, интегрируя Region Proposal Network для быстрого и точного предложения регионов интереса.

Глубокие сверточные нейронные сети пригодились в сегментации изображений. Fully Convolutional Networks заменили полносвязные слои на сверточные, и это позволило модели создавать карты классов для каждого пикселя на входном изображении. U-Net начал использовать симметричную структуру с кодирующими и декодирующими блоками, позволив модели выделять детализированные сегменты из низкоуровневых признаков. Mask R-CNN, расширение Faster R-CNN, добавило дополнительную ветку для предсказания масок объектов повысив точность инстансной сегментации.

1.6. Выводы по главе 1

В современном мире компьютерное зрение очень важно для многих вещей нашей жизни, начиная от распознавания лиц и заканчивая автономным вождением. В этой главе было обсуждено много аспектов компьютерного зрения, включая его актуальность и историю, а также основные направления и методы, такие как классификация, детекция объектов, сегментация изображений и отслеживание объектов.

Ученые со всего мира постоянно улучшают модели и методы, чтобы системы были точнее и работали лучше в разных условиях. Изучение машинного и глубокого обучения, а также разработка новых моделей и архитектур помогает решать сложные задачи и делает технологии лучше.

Компьютерное зрение помогает нам делать вещи быстрее и точнее, и добавляет в нашу жизнь много новых возможностей. Но обратная сторона медали, это защита данных и этические вопросы. Важно учитывать социальные последствия использования технологий компьютерного зрения в разных сферах нашей жизни.

Компьютерное зрение дает нам возможность улучшать жизнь людей в разных областях. В будущем нас ожидает много больших достижений и новых открытий, хотя это будущее уже становится настоящим.

ГЛАВА 2. ПОСТАНОВКА ЗАДАЧИ

2.1. Актуальность задачи

За последние десятилетия количество автомобилей на дорогах возросло в разы. Если в 2000 году в Беларуси на тысячу человек приходилось 138 автомобилей, то в 2019 эта цифра возросла до 329, и она продолжает расти. Из-за такого быстрого роста количества автомобилей на дорогах, появляется проблема мониторинга и управления дорожным движением для городов и стран. Быстрый рост числа автомобилей на дорогах ведет к росту транспортных заторов, увеличению количества дорожно-транспортных происшествий, ухудшению экологической обстановки и снижению качества жизни людей. Создание программных средств для анализа движения автомобильного транспорта на основе данных потокового видео – это возможное решение этой проблемы.

Из-за быстрого роста количества транспортных средств на дорогах появляется необходимость создания современных решений для управления дорожным движением. Имеющиеся методы контроля и управления трафиком оказываются не очень точными эффективными, так как они попросту не могут справиться с таким огромным и постоянно растущим объемом данных. По этой причине разработка программного средства, способного анализировать потоковое видео и предоставлять информацию о движении автомобилей получает первое место в приоритетности. Технологии компьютерного зрения и машинного обучения могут распознавать автомобили, отслеживать их перемещение, прогнозировать их поведение и выявлять возможные опасные ситуации на дорогах.

При разработке важно учитывать тот факт, что система будет работать в разных условиях съемки: начиная от разного уровня освещенности и заканчивая разными погодными условиями, то есть программа должна уметь адаптироваться к изменяющимся условиям и продолжать предоставлять точные данные.

Точность разработанного программного средства тесно связана с способностью объединяться с уже существующими системами управления дорожным движением. Это положительно скажется на улучшении мониторинга транспортных потоков и оптимизации работы светофоров. Использование таких решений – это большой шаг в улучшении дорожной инфраструктуры и повышении безопасности на дорогах.

Создание таких программных средств не просто техническая задача, а социально значимый проект, направленный на улучшение качества жизни людей, обеспечения безопасности дорожного движения и улучшения экологической обстановки. Создание и использование таких систем – это важный шаг на пути к умным городам будущего.

2.2. Цель и задачи исследования

Целью данной дипломной работы является разработка программного средства для анализа движения автомобильного транспорта на основе данных потокового видео. Из-за быстрорастущего количества автомобилей на дорогах страны, возникла необходимость в создании инструментов для мониторинга, анализа и управления дорожным движением. Программное средство, разрабатываемое в рамках данной дипломной работы направлено на решение этой проблемы путем использования технологий компьютерного зрения и машинного обучения. Для достижения поставленной цели необходимо решить несколько задач.

Первоначальная задача заключается в адаптации предобученной модели YOLO для работы с потоковым видео. Модель YOLO, которая известна своей высокой точностью и скоростью распознавания объектов, будет использована для идентификации и отслеживания транспортных средств в режиме реального времени.

Вторая задача заключается в создании системы мониторинга и анализа движения транспорта. Эта система должна уметь вести непрерывное наблюдение за дорожным движением, собирать и хранить данные. Система должна функционировать в разных условиях, начиная от городских дорог с плотным трафиком и заканчивая автомагистралями с высокими скоростями движения.

Разрабатываемое программное средство будет использоваться на городских дорогах и перекрестках, трассах и автомагистралях – везде, где есть автомобильное движение. Разработанное программное средство будет полезно для муниципальных служб и частных компаний, занимающихся мониторингом и анализом дорожного движения.

Таким образом, выполнение поставленной задачи направлено на создание точного инструмента для анализа движения автомобильного транспорта, способного работать в реальном времени в различных условиях. Выполнение поставленной задачи поможет улучшить управление дорожным движением и повысит безопасность и качество жизни в городах.

2.3 Ожидаемые результаты

В результате выполнения поставленной цели ожидается достижение результатов, которые улучшат управление движением автомобильного транспорта, которое будет точно анализировать потоковое видео для распознавания и отслеживания транспортных средств. С помощью модели YOLO будет достигнута высокая точность идентификации автомобилей.

Важной деталью данного проекта является возможность работы в реальном времени, благодаря чему станет возможным моментальное реагирование на изменения в дорожной обстановке, а также устранение возможных дорожно-транспортных происшествий.

Также немаловажной деталью является способность адаптироваться к различным условиям эксплуатации. Система должна быть устойчива к погодным и суточным изменениям.

Предполагается, что разработанная система будет объединена с существующими системами управления, что позволит улучшить уже имеющееся управление светофорными объектами, дорожными знаками и другими элементами дорожной инфраструктуры. Объединение с другими системами создаст комплексную систему управления трафиком и значительно улучшит дорожную обстановку.

Ожидается, что разработанная система положительно скажется на безопасности дорожного движения и экологической обстановке, путем

снижения количества таких явлений, как дорожно-транспортные происшествия, заторы и выбросы вредных веществ в атмосферу. Все это будет способствовать улучшению качества жизни людей и улучшению городской среды.

2.4. Выводы по главе 2

Из-за быстрого роста числа автомобилей на дорогах Беларуси появилась необходимость разработки решения для мониторинга и управления дорожным движением. Программное средство для анализа движения автомобилей станет удобным инструментом для решения этих проблем. Главная цель работы – это создание системы, которая будет использовать модель YOLO для идентификации и отслеживания автомобилей в реальном времени. Система будет работать в разных условиях проводя мониторинг и анализ дорожной обстановки. Ожидается что это решение повысит безопасность и улучшит управление трафиком, тем самым уменьшит количество заторов и аварий и улучшит экологическую обстановку. Интеграция с существующими системами управления дорожным движением позволит автоматизировать многие процессы, что в свою очередь создаст безопасную и экологически чистую городскую среду.

ГЛАВА 3. РЕШЕНИЕ ЗАДАЧИ ПОСТРОЕНИЯ НЕЙРОННОЙ СЕТИ ДЛЯ АНАЛИЗА ДВИЖЕНИЯ АВТОМОБИЛЬНОГО ТРАНСПОРТА

3.1. Реализация системы для анализа движения автомобильного транспорта на основе данных потокового видео

В этой главе будет подробно описан каждый шаг по созданию системы для анализа движения автомобильного транспорта. Первым делом следует импортировать в проект требующиеся библиотеки.

1. Импортирование необходимых библиотек.

```
import numpy as np
from ultralytics import YOLO
import cv2
import cvzone
import math
from sort import *
import torch
```

Листинг 3.1 Импорт библиотек

Примечание Источник: собственная разработка

В листинге 3.1 импортируются нужные для компьютерного зрения библиотеки, одна из них – библиотека Ultralytics.

Библиотека Ultralytics включает в себя инструменты для работы с компьютерным зрением, обнаружением объектов и сегментацией. Эта библиотека популярна благодаря своим моделям YOLO (You Only Look Once), которые на данный момент являются одними из самых точных моделей для задач обнаружения объектов. YOLO занимает лидирующие позиции в топе моделей благодаря своей высокой скорости и точности, а это то что нужно для применения в real time системах.

Библиотека содержит в себе предобученные модели, которые при необходимости можно дообучить на своих данных. Также эти модели очень легко интегрировать в различные проекты, так как библиотека имеет удобный функционал для разработчиков. Это позволяет разработчикам настраивать алгоритмы под конкретные задачи и данные, что сильно ускоряет процесс разработки и внедрения решений на базе компьютерного зрения.

```
video_capture = cv2.VideoCapture("dram_m_2.mp4")
```

Листинг 3.2 Открытие видеофайла "dram_m_2.mp4"

Примечание Источник: собственная разработка

Код (листинг 3.2) использует библиотеку **cv2** для захвата видеофайла. Функция **cv2.VideoCapture** из используется для открытия и управления видеофайлом "dram_m_2.mp4".



Рисунок 3.1 – Кадр из исходного видео

Примечание Источник: собственная разработка

```
codec = cv2.VideoWriter_fourcc(*'mp4v')
video_output = cv2.VideoWriter('output_video.mp4', codec, 30.0,
(int(video_capture.get(3)), int(video_capture.get(4))))
```

Листинг 3.3 Определение параметров для записи видео

Примечание Источник: собственная разработка

Данный код (листинг 3.3) выполняет создание объекта для записи видеофайла.

Здесь создается кодек FourCC (Four Character Code), который обозначает формат видеофайла, в данном данном случае используется кодек 'mp4v', что соответствует формату MPEG-4.

Аргументы в функции **cv2.VideoWriter** означают:

output_video.mp4: Имя выходного видеофайла.

fourcc: Кодек, определенный ранее.

30.0: Частота кадров для записываемого видео.

`(int(video_capture.get(3)), int(video_capture.get(4)))`: Ширина и высота кадра.

В результате этих операций создается готовый к использованию объект **VideoWriter**, который будет записывать видеок кадры в файл **output_video.mp4** с указанными параметрами.

```
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
print(f'Using device: {device}')
```

Листинг 3.4 Проверка доступности GPU

Примечание Источник: собственная разработка

В листинге 3.4 идет проверка доступности на компьютере графического процессора (GPU) с поддержкой CUDA для выполнения вычислений с использованием библиотеки PyTorch, и в зависимости от результата выбирается соответствующее устройство (GPU или CPU).

```
model = YOLO("../Yolo-Weights/yolov9c.pt").to(device)
```

Листинг 3.5 Загрузка модели YOLO

Примечание Источник: собственная разработка

Эта строка кода (листинг 3.6) загружает модель YOLO с предварительно обученными весами из файла **yolov9c.pt** и перемещает её на указанное устройство, обозначенное переменной **device**.

```
labels = ["person", "bicycle", "car", "motorbike", "bus",
"train", "truck", "traffic light"]
```

Листинг 3.6 Определение класса объектов

Примечание Источник: собственная разработка

Под переменную **labels** (листинг 3.6) создается список строк, где каждая строка представляет собой имя класса объектов, которые могут быть распознаны.

```
mask_image = cv2.imread("mask_m.png")
```

Листинг 3.7 Определение класса объектов

Примечание Источник: собственная разработка

Эта строка кода (листинг 3.7) загружает изображение с именем **mask_m.png** и сохраняет его в переменной **mask_image**. Маска показана на рисунке 3.2.

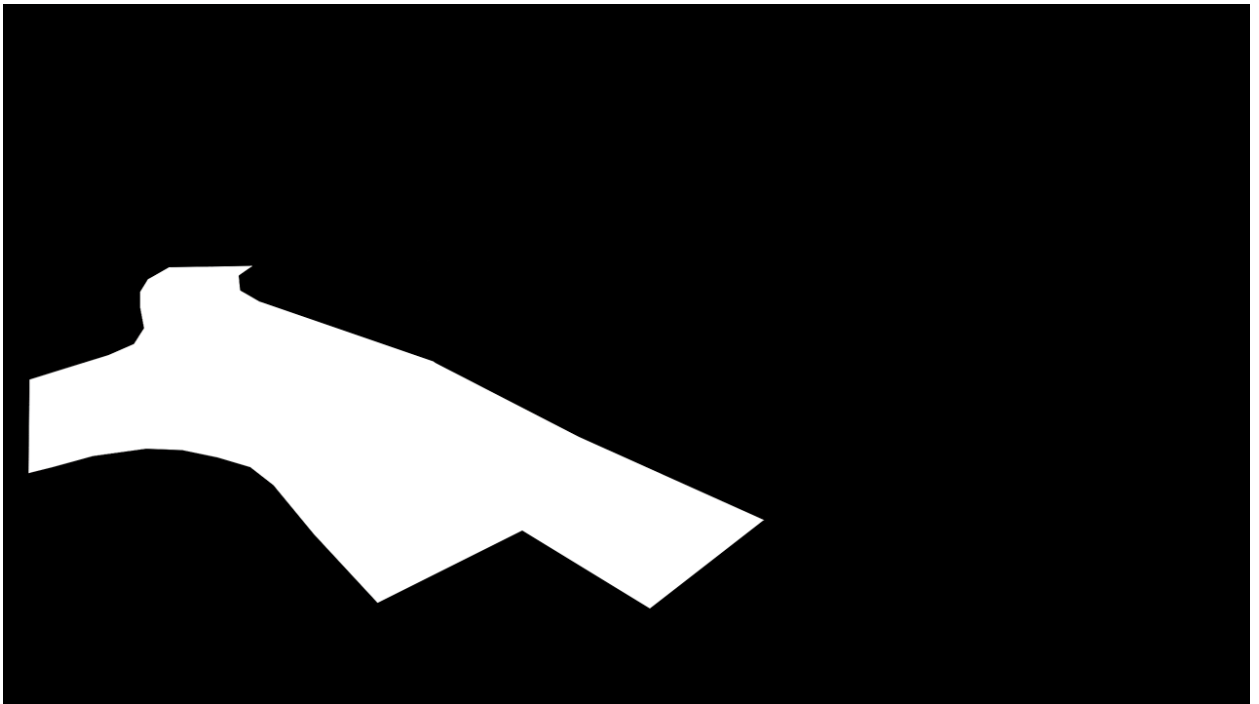


Рисунок 3.2 – Маска накладываемая на видео

Примечание Источник: собственная разработка

В последующем это изображение будет использоваться для наложения на видео и удаления лишнего из кадра.

```
object_tracker = Sort(max_age=15, min_hits=4,  
iou_threshold=0.3)
```

Листинг 3.8 Создание трекера с использованием алгоритма SORT

Примечание Источник: собственная разработка

Эта строка кода (листинг 3.8) создает объект **object_tracker** с использованием алгоритма SORT (Simple Online and Realtime Tracking) для отслеживания объектов в последовательности кадров из видео. Параметры **max_age=15**, **min_hits=4**, и **iou_threshold=0.3** настраивают поведение трекера:

max_age=15 задает максимальное количество кадров, в течение которых объект может отсутствовать и все еще быть отслеживаемым.

min_hits=4 указывает минимальное количество подтвержденных детекций для активации отслеживания объекта.

iou_threshold=0.3 устанавливает порог пересечения областей (Intersection over Union) для ассоциации детекций с существующими треками.

```
lines = [  
    [144, 315, 252, 301],  
    [428, 454, 539, 418],  
    [128, 352, 151, 457],  
    [332, 560, 465, 500]  
]  
  
count_tank = []  
count_books = []  
count_dram = []  
count_loft = []
```

Листинг 3.9 Обозначение координат линий и создание массивов для обнаруженных автомобилей

Примечание Источник: собственная разработка

В листинге 3.9 в переменной **lines** обозначены координаты линий в формате [x1, y1, x2, y2], которые будут отображаться во время мониторинга. Под переменными **count_** создаются массивы, в которые в дальнейшем будут добавляться уникальные ID автомобилей для последующего подсчета.

```

while True:
    ret, frame = video_capture.read()
    if not ret:
        break

    masked_frame = cv2.bitwise_and(frame, mask_image)
    detections_stream = model(masked_frame, stream=True)

    all_detections = np.empty((0, 5))

    for detection in detections_stream:
        boxes = detection.bboxes
        for box in boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            width, height = x2 - x1, y2 - y1
            confidence = round(box.conf[0].item(), 2)
            class_id = int(box.cls[0])
            label = labels[class_id]
            if label in ["car", "truck", "bus", "motorbike"]
and confidence > 0.3:
                detection_array = np.array([x1, y1, x2, y2,
confidence])
                all_detections = np.vstack((all_detections,
detection_array))

            tracked_objects = object_tracker.update(all_detections)

            for line in lines:
                cv2.line(frame, (line[0], line[1]), (line[2], line[3]),
(0, 0, 255), 2)

            for obj in tracked_objects:
                x1, y1, x2, y2, obj_id = map(int, obj)
                width, height = x2 - x1, y2 - y1
                cvzone.cornerRect(frame, (x1, y1, width, height), l=9,
rt=1, t=2, colorR=(255, 0, 255))
                cvzone.putTextRect(frame, f' {obj_id}', (max(0, x1),
max(35, y1)), scale=0.8, thickness=1, offset=1)

                center_x, center_y = x1 + width // 2, y1 + height // 2
                print(f'id: {obj_id} cx: {center_x}, cy: {center_y}')

```

```

        cv2.circle(frame, (center_x, center_y), 2, (0, 0, 255),
cv2.FILLED)

        if lines[0][0] < center_x < lines[0][2] and lines[0][1]
- 25 < center_y < lines[0][3] and obj_id not in count_tank:
            count_tank.append(obj_id)
            cv2.line(frame, (lines[0][0], lines[0][1]),
(lines[0][2], lines[0][3]), (0, 255, 0), 2)
            if lines[1][0] < center_x < lines[1][2] and lines[1][3]
- 10 < center_y < lines[1][1] and obj_id not in count_dram:
                count_dram.append(obj_id)
                cv2.line(frame, (lines[1][0], lines[1][1]),
(lines[1][2], lines[1][3]), (0, 255, 0), 2)
                if lines[2][0] < center_x < lines[2][2] and lines[2][1]
< center_y < lines[2][3] and obj_id not in count_books:
                    count_books.append(obj_id)
                    cv2.line(frame, (lines[2][0], lines[2][1]),
(lines[2][2], lines[2][3]), (0, 255, 0), 2)
                    if lines[3][0] < center_x < lines[3][2] and lines[3][3]
< center_y < lines[3][1] and obj_id not in count_loft:
                        count_loft.append(obj_id)
                        cv2.line(frame, (lines[3][0], lines[3][1]),
(lines[3][2], lines[3][3]), (0, 255, 0), 2)

        print(f'Tank: {count_tank}', f'Books: {count_books}',
f'Dram: {count_dram}', f'Loft: {count_loft}', sep='\n')

        cvzone.putTextRect(frame, f'Tank: {len(count_tank)}', (190,
585), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
        cvzone.putTextRect(frame, f'Dram: {len(count_dram)}', (190,
615), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
        cvzone.putTextRect(frame, f'Books: {len(count_books)}',
(190, 645), 0.8, 1, (0, 0, 0), (255, 255, 255),
cv2.FONT_ITALIC)
        cvzone.putTextRect(frame, f'Loft: {len(count_loft)}', (190,
675), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)

        cv2.imshow("God's Eye", frame)
        video_output.write(frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

```

Листинг 3.10 Главный цикл программы

Примечание Источник: собственная разработка

Разберем подробно каждую часть кода из листинга 3.10:

```
ret, frame = video_capture.read()
if not ret:
    break
```

Листинг 3.11 Чтение кадра из видео

Примечание Источник: собственная разработка

Функция **video_capture.read()** читает кадр из видео. Если кадр не удалось прочитать, то есть **ret** равен **False**, цикл **while** прерывается.

```
masked_frame = cv2.bitwise_and(frame, mask_image)
detections_stream = model(masked_frame, stream=True)
```

Листинг 3.12 Создание маски изображения

Примечание Источник: собственная разработка

Переменная **masked_frame** создается путем наложения маски **mask_image** на текущий кадр **frame** с помощью побитовой операции AND.

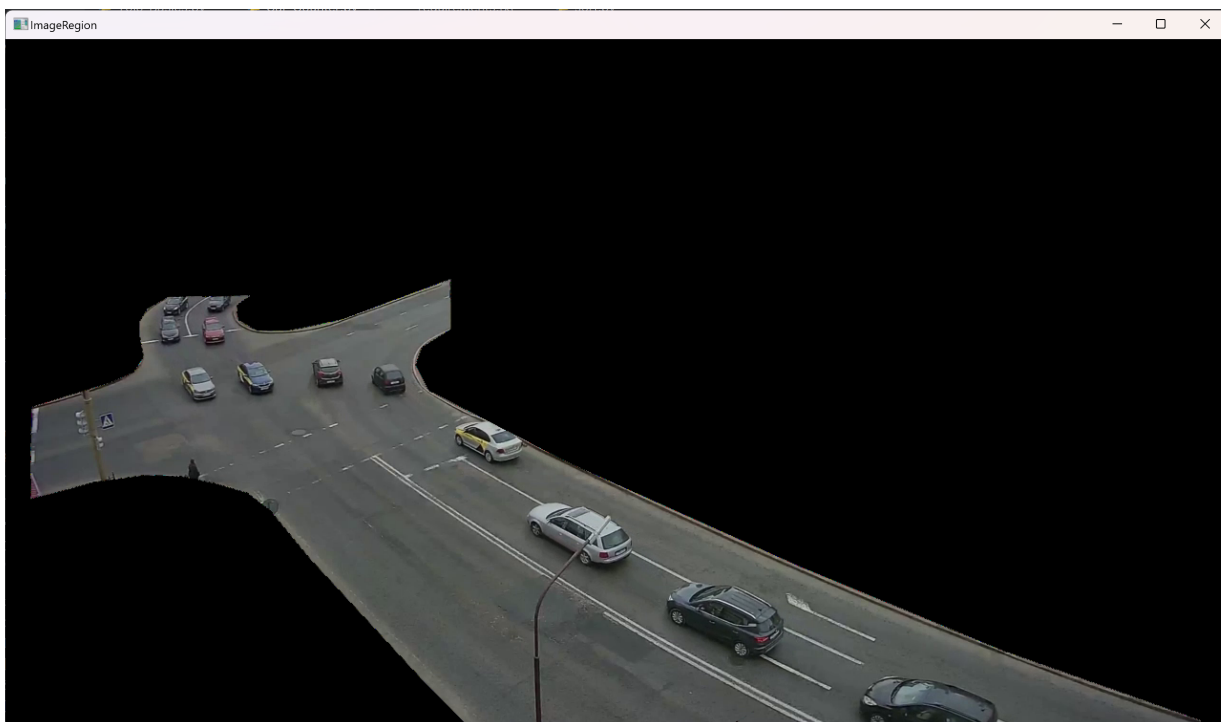


Рисунок 3.3 – Демонстрация наложения маски на исходное видео

Примечание Источник: собственная разработка

Переменная **detections_stream** содержит в себе результаты обнаружения объектов, полученные с помощью модели YOLO.

```
all_detections = np.empty((0, 5))
```

Листинг 3.13 Подготовка массива для обнаружений

Примечание Источник: собственная разработка

Переменная **all_detections** инициализируется как пустой массив для хранения данных о найденных объектах.

```
for detection in detections_stream:
    boxes = detection.bboxes
    for box in boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        width, height = x2 - x1, y2 - y1
        confidence = round(box.conf[0].item(), 2)
        class_id = int(box.cls[0])
        label = labels[class_id]
        if label in ["car", "truck", "bus", "motorbike"] and
confidence > 0.3:
            detection_array = np.array([x1, y1, x2, y2,
confidence])
            all_detections = np.vstack((all_detections,
detection_array))
```

Листинг 3.14 Обработка результатов модели

Примечание Источник: собственная разработка

Переменная **detections_stream** содержит все результаты, возвращаемые моделью для текущего кадра. Программа обходит каждый элемент **detections** в переменной **detections_stream**.

В каждом **detections** есть атрибут **bboxes**, который содержит все обнаруженные рамки для этого результата. Переменная **bboxes** – это список, который содержит в себе каждую обнаруженную рамку.

С помощью цикла **for** обходим каждую рамку **box** в **bboxes**.

Переменная **box.xxyy[0]** содержит координаты углов рамки в формате [x1, y1, x2, y2]. Эти координаты представляют верхний левый (x1, y1) и нижний правый (x2, y2) углы рамки.

Преобразуем координаты в целые числа **int**.

Далее вычисляем ширину **width** и высоту **height** рамки.

Внутри переменной **box.conf[0]** содержится значение доверительности для данной рамки. Доверительность умножается на 100, округляется вверх до ближайшего целого числа и делится на 100 чтобы получить значение с двумя знаками после запятой.

В переменной **box.cls[0]** содержится индекс класса для данной рамки. Программа конвертирует индекс класса в целое число **int** и использует индекс для получения имени класса из списка **labels**.

Далее проверяется, принадлежит ли текущий объект к обозначенным ранее классам (**car**, **truck**, **bus**, **motorbike**) и превышает ли доверительность объекта пороговое значение 0.3.

Если объект прошел фильтрацию, создается массив **detection_array**, содержащий координаты рамки и доверительность. Добавляем **detection_array** в массив **all_detections** с помощью **np.vstack**.

Этот процесс выполняется для всех обнаруженных объектов в текущем кадре, в результате чего получаем массив **all_detections**, который содержит данные обо всех интересующих объектах, обнаруженных в этом кадре. Затем этот массив используется для обновления трекера.

```
tracked_objects = object_tracker.update(all_detections)
```

Листинг 3.15 Обновление трекера

Примечание Источник: собственная разработка

Переменная **object_tracker** это объект класса **Sort**, используемый для отслеживания объектов между кадрами видео.

Метод **update** принимает на вход массив **all_detections**, содержащий информацию о всех обнаруженных объектах в текущем кадре.

Переменная **all_detections** это массив размером Nx5, где N это количество обнаруженных объектов. Каждый ряд содержит: [x1, y1, x2, y2, conf].

Метод **update** возвращает массив **tracked_objects**, содержащий информацию о трекаемых объектах и их идентификаторы.

В листинге 3.15 вызов **object_tracker.update(all_detections)** обновляет состояние трекера на основе текущих обнаружений и возвращает обновленные позиции и идентификаторы треков. Далее **tracked_objects** будет

содержать в себе массив с информацией о трекаемых объектах, включая их обновленные координаты и уникальные идентификаторы, что дальше будет использоваться для подсчета объектов.

Как работает трекер SORT

Трекер SORT (Simple Online and Realtime Tracking) использует алгоритм для отслеживания объектов в видео, сочетающий в себе метод ассоциации объектов (Data Association) с использованием метода Калмана и линейного предсказания для определения местоположения объектов в последующих кадрах.

Важные параметры трекера SORT

max_age: Максимальное количество кадров, в течение которых трек может оставаться неактивным, прежде чем он будет удален.

min_hits: Минимальное количество детекций, прежде чем трек будет считаться надежным и начнет отображаться.

iou_threshold: Порог IOU для сопоставления обнаружений с треками. Если IOU между обнаружением и треком ниже этого порога, они не будут сопоставлены.

```
for line in lines:
    cv2.line(frame, (line[0], line[1]), (line[2], line[3]), (0,
0, 255), 2)
```

Листинг 3.16 Отрисовка линий на изображении

Примечание Источник: собственная разработка

С помощью цикла `for` отрисовываем каждую линию, координаты которой находятся во вложенном списке списка **lines**.

Функция **cv2.line** в листинге 3.16 используется для рисования линий на изображении и принимает следующие параметры:

frame: Изображение, на котором будет нарисована линия.

pt1: Начальная точка линии (x1, y1), то есть **line[0]**, **line[1]**.

pt2: Конечная точка линии (x2, y2), то есть **line[2]**, **line[3]**.

color: Цвет линии в формате BGR.

thickness: Толщина линии.

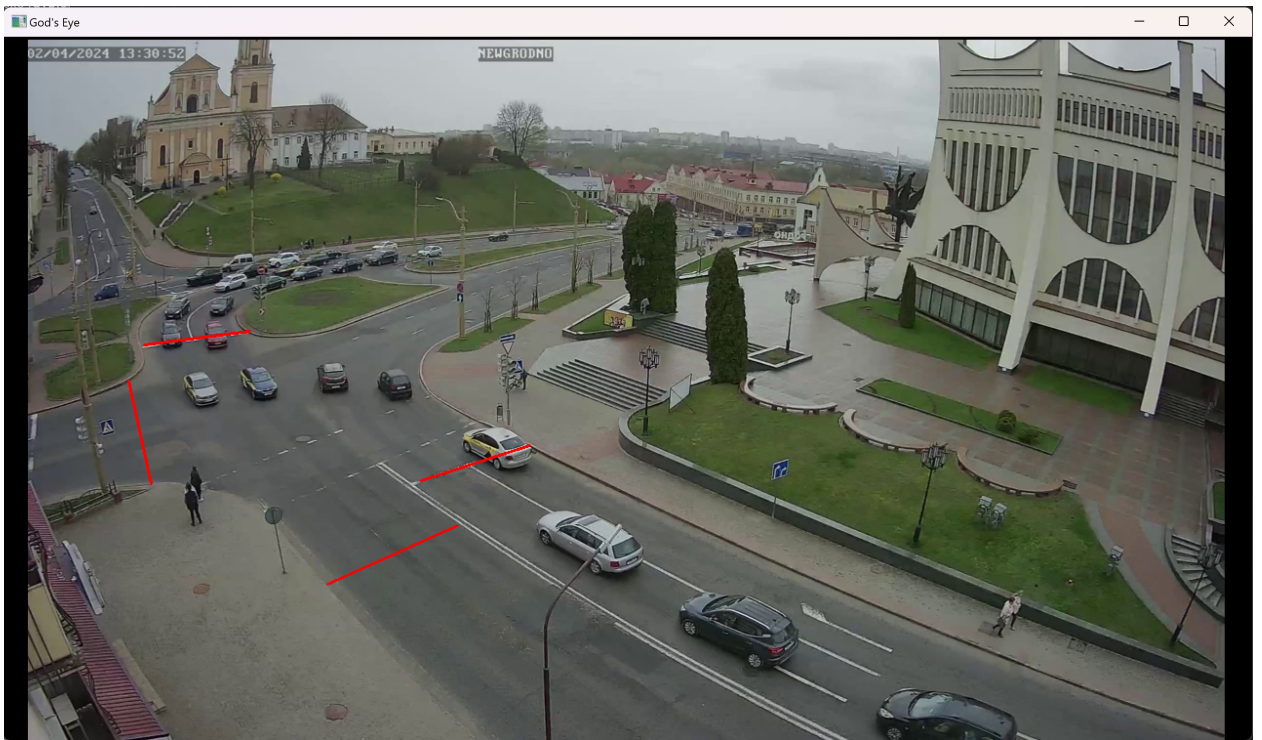


Рисунок 3.4 – Результат отрисовки линий на изображении

Примечание Источник: собственная разработка

```
for obj in tracked_objects:
    x1, y1, x2, y2, obj_id = map(int, obj)
    width, height = x2 - x1, y2 - y1
    cvzone.cornerRect(frame, (x1, y1, width, height), l=9,
rt=1, t=2, colorR=(255, 0, 255))
    cvzone.putTextRect(frame, f' {obj_id}', (max(0, x1),
max(35, y1)), scale=0.8, thickness=1, offset=1)

    center_x, center_y = x1 + width // 2, y1 + height // 2
    print(f'id: {obj_id} cx: {center_x}, cy: {center_y}')
    cv2.circle(frame, (center_x, center_y), 2, (0, 0, 255),
cv2.FILLED)
```

Листинг 3.17 Обработка результатов трекера

Примечание Источник: собственная разработка

Переменная **tracked_objects** это массив возвращаемый трекером, который содержит в себе координаты и идентификаторы объектов, отслеживаемых в текущем кадре. Цикл проходит по каждому элементу **obj** в **tracked_objects**.

Далее извлекаем координаты углов рамки (x_1 , y_1) и (x_2 , y_2), а также идентификатор `obj_id`. Преобразуем координаты в целые числа и вычисляем ширину **width** и высоту **height** рамки.

Используем функцию **cvzone.cornerRect** для рисования рамки вокруг объекта. Эта функция включает в себя следующие параметры:

l=9 задает длину углов рамки.

rt=1 задает радиус скругления углов.

t=2 задает толщину линии.

colorR=(255, 0, 255) задает цвет линии, в данном случае розовый.

Далее используем функцию **cvzone.putTextRect** для отображения идентификатора объекта рядом с рамкой.

Вычисляем координаты центра объекта **center_x**, **center_y** и рисуем красный круг в центре объекта для визуализации. Этот круг будет точкой, которая будет сигналом для увеличения счетчика в случае, если точка пересечет нарисованную линию.

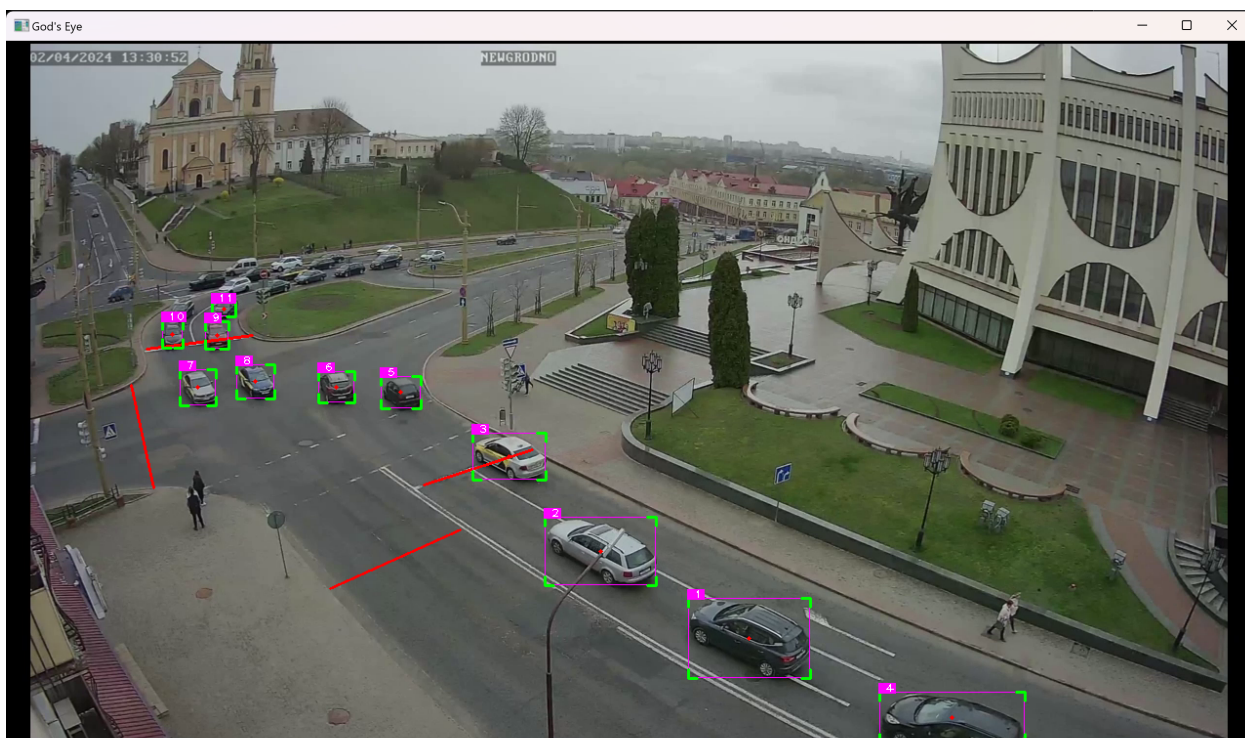


Рисунок 3.5 – Демонстрация обработки результатов трекера

Примечание Источник: собственная разработка

```

if lines[0][0] < center_x < lines[0][2] and lines[0][1] - 25 <
center_y < lines[0][1] and obj_id not in count_tank:
    count_tank.append(obj_id)
    cv2.line(frame, (lines[0][0], lines[0][1]), (lines[0][2],
lines[0][3]), (0, 255, 0), 2)
if lines[1][0] < center_x < lines[1][2] and lines[1][3] - 10 <
center_y < lines[1][1] and obj_id not in count_dram:
    count_dram.append(obj_id)
    cv2.line(frame, (lines[1][0], lines[1][1]), (lines[1][2],
lines[1][3]), (0, 255, 0), 2)
if lines[2][0] < center_x < lines[2][2] and lines[2][1] <
center_y < lines[2][3] and obj_id not in count_books:
    count_books.append(obj_id)
    cv2.line(frame, (lines[2][0], lines[2][1]), (lines[2][2],
lines[2][3]), (0, 255, 0), 2)
if lines[3][0] < center_x < lines[3][2] and lines[3][3] <
center_y < lines[3][1] and obj_id not in count_loft:
    count_loft.append(obj_id)
    cv2.line(frame, (lines[3][0], lines[3][1]), (lines[3][2],
lines[3][3]), (0, 255, 0), 2)

```

Листинг 3.18 Проверка пересечения линий и подсчет объектов

Примечание Источник: собственная разработка

В листинге 3.18 проверяем, пересекает ли центр объекта одну из заданных линий

Если центр объекта в пределах координат линии, определенной переменной **lines** и объект еще не был учтен **obj_id not in count_**, тогда добавляем его идентификатор в **count_**. Для визуализации пересечения перерисовываем линию зеленым цветом.

Такая же процедура повторяется для остальных переменных **lines**.

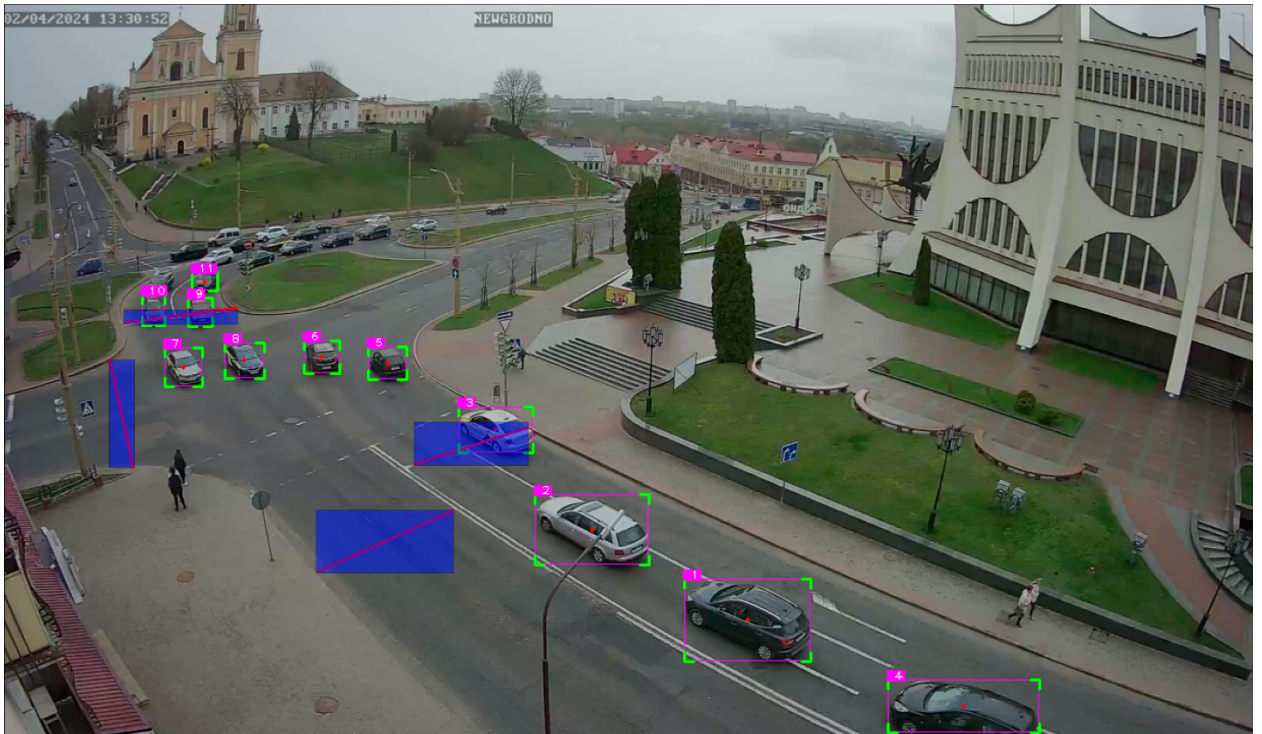
Рисунок 3.6 – Демонстрация зон ведущих подсчет объектов

Примечание Источник: собственная разработка

```

print(f'Tank: {count_tank}', f'Books: {count_books}', f'Dram:
{count_dram}', f'Loft: {count_loft}', sep='\n')

```



Листинг 3.19 Вывод текущего подсчета объектов

Примечание Источник: собственная разработка

Функция **print** в листинге 3.19 выводит текущий список объектов для каждой зоны. Она не является важной частью программы и служит отчетом работы во время выполнения программы.

```
cvzone.putTextRect(frame, f'Tank: {len(count_tank)}', (190,
585), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
cvzone.putTextRect(frame, f'Dram: {len(count_dram)}', (190,
615), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
cvzone.putTextRect(frame, f'Books: {len(count_books)}', (190,
645), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
cvzone.putTextRect(frame, f'Loft: {len(count_loft)}', (190,
675), 0.8, 1, (0, 0, 0), (255, 255, 255), cv2.FONT_ITALIC)
```

Листинг 3.20 Отображение подсчета на изображении

Примечание Источник: собственная разработка

С помощью **cvzone.putTextRect** ображаем подсчет объектов для каждой зоны на изображении **frame**. С помощью этой функции на изображении в углу слева будет отображаться количество автомобилей, пересекших линию.

С помощью функции **cv2.imshow** открывается окно с изображением **frame**.

Далее через функцию **video_output.write** записывается текущий кадр в выходной видеофайл. Этот метод используется в случае, если нужно сохранить обработанное видео.

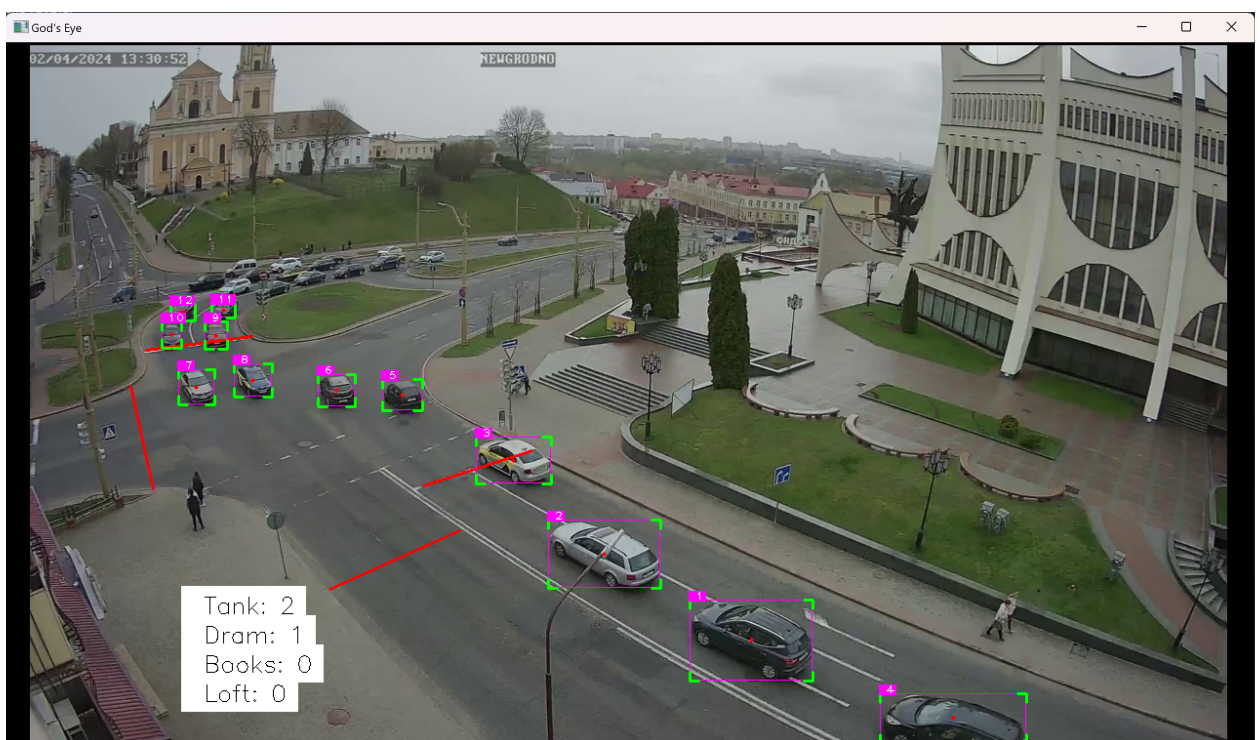


Рисунок 3.7 – Демонстрация обработанного кадра из видео

Примечание Источник: собственная разработка

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

Листинг 3.21 Прерывание цикла по нажатию клавиши 'q'

Примечание Источник: собственная разработка

Для завершения программы используем метод `cv2.waitKey`. Если нажата клавиша 'q', цикл прерывается.

```
video_capture.release()  
video_output.release()  
cv2.destroyAllWindows()
```

Листинг 3.22 Освобождение ресурсов

Примечание Источник: собственная разработка

Вызов метода **`release()`** в листинге 3.22 освобождает видеозахват и освобождает видеозапись.

Это действие завершает все операции чтения из видеофайла и освобождает системные ресурсы, удерживаемые объектом **`video_capture`**.

Метод **`cv2.destroyAllWindows()`** закрывает все окна, открытые функцией **`cv2.imshow`**.

3.2. Выводы по главе 3

В третьей главе поставленные задачи к разработке программных средств для анализа движения автомобильного транспорта на основе данных потокового видео были выполнены. В ходе выполнения работы была реализована программа, которая распознает автомобильный транспорт, анализирует его движение и собирает данные для последующего анализа.

В данной главе была подробно описана структура программы, этапы ее настройки. Полученные результаты подтверждают выполнение поставленных задач и гарантируют эффективную работу программного средства в условиях реального мира.

ЗАКЛЮЧЕНИЕ

В процессе выполнения данной дипломной работы была проведена обширная исследовательская работа по разработке программных средств для анализа движения автомобильного транспорта на основе данных потокового видео. Начиная с обзора литературы, где были рассмотрены существующие

методы в области компьютерного зрения и машинного обучения, до конкретных шагов настройки модели. Каждый этап исследования направлен на достижение общей цели – создания системы анализа движения автомобилей.

Особое внимание уделялось не только выбору модели, но и её подготовке, поскольку качество модели напрямую влияет на производительность и точность анализа. Во время подготовки модели были выполнены этапы сбора и обработки данных, настройка и оптимизация модели для работы с потоковым видео.

Созданное программное средство улучшит дорожную инфраструктуру, повысит безопасность на дорогах общего пользования, уменьшит пробки на дорогах и снизит количество дорожно-транспортных происшествий. В конце нужно подчеркнуть, что созданное решение имеет возможность объединяться с другими системами анализа движения транспорта, что в будущем послужит улучшению качества транспортной инфраструктуры.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Сверточная нейронная сеть [Электронный ресурс] Режим доступа: <https://intellect.ml/svertochnaya-nejronnaya-set-convolutional-neural-network-cnn-6013/> - Дата доступа: 01.12.2023.

2. Интернет-ресурс Wikipedia [Электронный ресурс] / Интернет-портал. – Режим доступа: https://ru.wikipedia.org/wiki/Машинное_обучение . - Дата доступа: 01.12.23.
3. Интернет-ресурс GitHub [Электронный ресурс] / Интернет-портал. – Режим доступа: <https://github.com/ultralytics/ultralytics> - Дата доступа: 01.12.23.
4. Лекции по нейронным сетям [Электронный ресурс] Режим доступа: <https://www.intuit.ru/studies/courses/6/6/lecture/178> - Дата доступа: 01.12.2023
5. Интернет-ресурс Wikipedia [Электронный ресурс] / Интернет-портал. – Режим доступа: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)) - Дата доступа: 01.12.23.
6. Интернет-ресурс Wikipedia [Электронный ресурс] / Интернет-портал. – Режим доступа: https://ru.wikipedia.org/wiki/Компьютерное_зрение - Дата доступа: 01.12.23.
7. Интернет-ресурс universe.roboflow [Электронный ресурс] / Интернет-портал. – Режим доступа: <https://universe.roboflow.com/k-qd1zc/car-model-detect2> - Дата доступа: 01.12.23.
8. Применение нейронных сетей для задач классификации [Электронный ресурс] / Интернет-портал. – Режим доступа: <https://basegroup.ru/community/articles/classification> - Дата доступа: 01.12.2023.